

RTOS THS
开发指南

版本号: 1.0
发布日期: 2023.08.29

深圳易新泰科技有限公司YiXinTai

版本历史

版本号	日期	制/修订人	内容描述
1.0	2023.08.29	AWA1812	初始版本

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 概述 **1**

1.1 文档简介 . . . 1
1.2 目标读者 . . . 1
1.3 适用范围 . . . 1
1.4 术语说明 . . . 1
1.5 文档约定 . . . 2
1.5.1 标志说明 . . . 2
1.5.2 地址与数据描述方法约定 . . . 2
1.5.3 数值单位约定 . . . 2

2 模块介绍 **4**

2.1 模块功能介绍 . . . 4
2.2 模块配置介绍 . . . 4
2.2.1 menuconfig 配置说明 . . . 4
2.2.2 sysconfig 配置说明 . . . 4
2.3 源码结构介绍 . . . 4
2.4 模块框架介绍 . . . 5
2.5 结构体介绍 . . . 5
2.6 接口介绍 . . . 6
2.6.1 thermal_init . . . 6
2.6.2 thermal_zone_device_register . . . 7
2.6.3 thermal_zone_device_unregister . . . 7
2.6.4 thermal_external_get_zone_number . . . 7
2.6.5 thermal_external_get_temp . . . 7

3 调试使用 **9**

3.1 命令说明 . . . 9

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 概述

1.1 文档简介

本文介绍温控模块的框架及其使用方法。

1.2 目标读者

温控框架编写、维护人员，应用开发人员

1.3 适用范围

表 1-1: 适用产品列表

产品名称	内核版本	源文件
R128	FreeRTOS	lichee/rtos-components/aw/thermal*
XR875	FreeRTOS	lichee/rtos-components/aw/thermal*

1.4 术语说明

表 1-2: 术语说明

术语	解释说明
THS	Thermal 子系统，负责温度管理

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1.5 文档约定

1.5.1 标志说明

⚠ 注意

- **提醒操作中应注意的事项。不当的操作可能会损坏器件，影响可靠性、降低性能等。**

说明

为准确理解文中指令、正确实施操作而提供的补充或强调信息。

技巧

一些容易忽视的小功能、技巧。了解这些功能或技巧能帮助解决特定问题或者节省操作时间。

1.5.2 地址与数据描述方法约定

本文档在描述地址、数据时遵循如下约定：

表 1-3: 地址与数据描述方法约定

符号	例子	说明
0x	0x0200，0x79	地址或数据以 16 进制表示。
0b	0b010，0b00 000 111	数据采用二进制表示 (寄存器描述除外)。
X	00X，XX1	数据描述中，X 代表 0 或 1。 例如，00X 代表 000 或 001；XX1 代表 001，011，101 或 111。

1.5.3 数值单位约定

本文档在描述数据容量（如 NAND 容量）时，单位词头代表的是 1024 的倍数；描述频率、数据速率等时则代表的是 1000 的倍数。具体如下：

表 1-4: 数值单位约定

类型	符号	对应数值
数据容量（如 NAND 容量）	1 K	1024
	1 M	1 048 576
	1 G	1 073 741 824
频率，数据速率等	1 k	1000

版权所有 © 珠海全志科技股份有限公司。保留一切权利

类型	符号	对应数值
	1 M	1 000 000
	1 G	1 000 000 000

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 模块介绍

2.1 模块功能介绍

THS 是温度控制系统，THS 提供使用 sensor 测量当前温度的功能，通常还能使用 IPA(Intelligent Power allocator)、step-wise 等温控策略算法调控温度，一般是通过调频来影响温度，而 sensor 则是被安置在 cpu、gpu 等关键位置处。此外，THS 提供过温保护功能，若检测到温度达到过温温度将会停止系统运转。

rtos 上 THS 模块当前无温控策略实现，仅提供温度检测与过温保护功能。

2.2 模块配置介绍

2.2.1 menuconfig 配置说明

```
System components  --->
 aw components  --->
  [*] Thermal Support
```

2.2.2 sysconfig 配置说明

```
;-------------------------------------------------------------------------
;thermal configuration
;-------------------------------------------------------------------------
[thermal]
critical_temperature = 105000    # 过温温度，105000表示105摄氏度，当检测到温度高于该设置值时，触发过温保护
```

2.3 源码结构介绍

```
lichee/rtos-components/aw/thermal
 ├──── Kconfig
 ├──── Makefile
 ├──── thermal_core.c    # 温控框架代码
 ├──── thermal_external.h
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
├── thermal.h
├── thermal_sensor.c   # sensor代码，提供平台sensor的注册、读温、校准动作
└── thermal_sensor.h
```

2.4 模块框架介绍

rtos 上 THS 包含两个组件：thermal zone（温度域）以及 thermal zone device（获取温度的设备）。

系统运行时，由一个最高优先级的 THS task 周期性通过获取温度的设备为每个温度域更新温度值，获取温度后进行温度处理。处理包含两步，一是与过温温度比较，若获取到的温度大于等于过温温度，则触发关机，二是将获取温度作为参数运行算法计算目标温度，随后通过调频动作调控温度，当前温度处理只进行到第一步。此外，可以通过写入模拟温度值，替换每次获取到的温度值，用于测试。

thermal zone 即温度域，用于描述一组设备的温度特性，共享传感器、温度值即过温温度配置，由同一温控策略管控，是 THS 运转时的监管对象。

thermal zone device 是获取温度的设备的抽象，测温设备多种多样，使用温度传感器、温敏电阻、GPADC 等都可以测量温度，这些设备可以在实现温度校准、温度读取的方法后，将自身注册给 thermal zone，在 THS 运转时提供获取温度域温度的方法，温度校准用于减少测量温度与实际温度的偏差，具体动作与平台相关，由平台提供。

2.5 结构体介绍

- THS 框架

```
struct thermal_zone_device_ops {          # 获取温度的设备的回调
   int (*get_temp)(void *data, int *temp);   # 温度获取实现
   void (*shutdown)(void);    # 关机实现
};

struct thermal_zone_device {          # 描述获取温度的设备
   void *priv;          # 私有数据
   TaskHandle_t  xHandle;
   QueueHandle_t xQueue;
   struct thermal_zone_device_ops *ops;
   struct thermal_zone *zone;       # 隶属温度域
};

struct thermal_zone_ops {             # 温度域回调
   int (*set_emul_temp)(void *data, const int emul_temp); # 设置模拟温度
   int (*set_crit_temp)(void *data, const int crit_temp);    # 设置过温温度
   int (*set_func_switch)(void *data, const int func_switch_mask); # 设置启用的功能（模拟温度及过温保护功能）
};

struct thermal_zone {          # 描述温度域
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    char *type;         # 名称
    char mode;          # 运行状态，是否启用
    int id;             # 标识ID
    int last_temperature;       # 上轮检测温度
    int temperature;            # 当前温度
    int emul_temperature;       # 模拟温度
    int crit_temperature;       # 过温温度
    int func_switch;            # 启用功能
    struct thermal_zone_ops *ops;
    struct thermal_zone_device *zone_dev;     # 使用的测温设备
    unsigned int polling_interval;      # 检测动作周期
    osal_timer_t polling_timer;
    hal_mutex_t lock;
    struct list_head node;
};
```

- 平台 sensor

sensor 指代平台的测温模块抽象，由 sensor 注册温度域并提供温度获取设备。

```
struct thermal_sensor_chip {            # 描述sensor属性
    char *name;         # 名称
    char **zone_name;           # 将注册的温度域名称

    int sensor_num;         # 平台温度域数量
    int slope;              # 校准用斜率
    int intercept;          # 校准用截距
    int temp_conpensation;      # 补偿温度

    int (*init)(struct thermal_sensor_chip *ths);
    int (*deinit)(struct thermal_sensor_chip *ths);
    int (*calibrate)(struct thermal_sensor_chip *ths);    # 温度校准实现
    int (*get_temp)(struct thermal_sensor_chip *ths, int *temp);   # 获取温度实现

    struct ths_device *ths_dev;
};

struct ths_device {                 # 描述一个sensor
    struct thermal_sensor_chip *ths;            # sensor属性
    struct thermal_zone_device *tzd[THERMAL_MAX_SENSOR_NUM];   # 包含的获取温度设备
};
```

2.6 接口介绍

2.6.1 thermal_init

- 原型：

```
int thermal_init(void)
```

- 功能：初始化 thermal 框架

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 返回值：返回 0 表示初始化成功，其余值表示初始化失败

2.6.2 thermal_zone_device_register

- 原型：

```
struct thermal_zone_device *thermal_zone_device_register(char *zone_name,
        struct thermal_zone_device_ops *ops, void *priv);
```

- 功能：注册一个 thermal zone device，同时生成温度域
- 返回值：返回 thermal zone device 句柄，返回 NULL 表示注册失败
- 参数：

```
@zone_name：想生成的温度域名称
@ops：thermal zone device动作回调
@priv：私有数据
```

2.6.3 thermal_zone_device_unregister

- 原型：

```
int thermal_zone_device_unregister(struct thermal_zone_device *zone_dev)
```

- 功能：移除一个 thermal zone device 及其隶属温度域
- 返回值：返回 0 表示移除成功，其余值表示移除失败
- 参数：注册时返回的 thermal zone device 句柄

2.6.4 thermal_external_get_zone_number

- 原型：

```
int thermal_external_get_zone_number(void)
```

- 功能：获取当前的温度域个数
- 返回值：温度域个数

2.6.5 thermal_external_get_temp

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
int thermal_external_get_temp(int *id_buf, int *temp_buf, int num)
```

- 功能：获取当前所有温度域的温度
- 返回值：返回 0 表示获取成功，其余值表示获取失败
- 参数：

```
@id_buf：id缓冲区，将被填入当前温度域的ID
@temp_buf：温度值缓冲区，将被填入与ID相对应得温度域温度
@num：温度域个数
```

深圳易新泰科技有限公司YiXinTai

3 调试使用

3.1 命令说明

```
thermal -h

Usage Help:
-c: set critical temperature.     # 设置过温温度，例如设置105摄氏度：thermal -c 105000
-e: set emulation temperature.    # 设置模拟温度，例如设置105摄氏度：thermal -c 105000
-f: disable/enable function:      # 设置启动功能
   0x1 - Over-temperature protection; 0x2 - emulation temperature;
   enable all function: thermal -f 3
-g: get current temperature.      # 打印所有温度域温度
-l: set thermal log level:        # 设置THS的日志等级，若使能了DBG日志，每次更新温度都会输出当前温度与上一轮温度值，可
       用于观察变化
   0x1 - ERR, 0x2 - WARN, 0x4 - DBG, 0x8 - INFO;
   open all logs: thermal -l 15
-h: print Usage Help.
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2023 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明


、 
、全志科技、  （不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。